COURSE PROJECT — FIRST REVIEW (OCTOBER 12, 2026)

Digital Logic Circuit Simulator

A Graph-Theoretic Framework for Combinational Logic Validation & Simulation

Presented by:

- **Markanda Meher** (Roll No. 206126020)
- **Nikhil** (Roll No. 206126021)

Department of Computer Science & Engineering | Advanced Data Structures & Algorithms

REVIEW POINT 1

1. Introduction — Problem Statement & Need for Solution

The Problem in Physical Prototyping

Breadboard Clutter & Tangled Wires: Constructing multi-gate logic circuits on physical breadboards leads to dense jumper-wire tangles, loose pin connections, and unreliable contact resistance.

Tedious Fault Localization: Debugging broken traces or faulty gate inputs requires time-consuming manual probing using multimeters or logic probes.

Hardware Damage Risks: Accidental short circuits, inverted IC pins, or incorrect VCC/GND connections permanently damage physical 7400-series TTL integrated circuits.

Poor Scalability: Scaling beyond 10-15 gates becomes physically cumbersome and impractical for student laboratory experiments.

No Automated Design Checking: Physical boards offer no mechanism to detect unintended feedback loops or race conditions before power is applied.

The Need for Software Simulation

Safe, Zero-Cost Experimentation: Eliminates physical component wear, damaged ICs, and lab resource bottlenecks while enabling instant circuit restructuring.

Automated Cycle Detection: Models the circuit as a Directed Graph; executes linear-time graph algorithms to identify and halt illegal feedback loops.

Deterministic Topological Scheduling: Formally guarantees that upstream driver signals settle before downstream load gates evaluate, preventing race conditions.

Exhaustive Truth Table Verification: Automatically tests all 2^N input permutations, verifying circuit behavior against theoretical Boolean algebra in milliseconds.

Pedagogical Value: Demonstrates the direct practical application of core graph algorithms (DFS, Kahn's BFS, Disjoint Sets) to solve computer architecture problems.

REVIEW POINT 2

2. Literature Survey — Analysis of 3 Papers & Their Drawbacks

Paper 1: A. B. Kahn (1962) — "Topological Sorting of Large Networks"

Communications of the ACM, Vol. 5, No. 11, pp. 558–562

Analysis / Contribution: Introduced in-degree based queue processing (BFS style) to establish a linear processing order in Directed Acyclic Graphs with optimal O(V + E) runtime complexity.

Drawbacks: (1) Does not locate cycle paths: when cycles exist, in-degrees fail to reach zero and the algorithm halts without isolating the offending back-edges. (2) Arbitrary tie-breaking: does not produce levelized evaluation stages for parallel execution.

Paper 2: R. E. Tarjan (1972) — "Depth-First Search and Linear Graph Algorithms"

SIAM Journal on Computing, Vol. 1, No. 2, pp. 146–160

Analysis / Contribution: Formulated 3-state depth-first search (White/Gray/Black node coloring) and strongly connected components (SCC) analysis running in linear O(V + E) time.

Drawbacks: (1) Recursion call-stack overflow risk on deep or large netlists. (2) Identifying SCCs or back-edges detects cycles but does not provide an execution schedule for signal simulation.

Paper 3: R. E. Tarjan (1975) — "Efficiency of a Good But Not Linear Set Union Algorithm"

Journal of the ACM, Vol. 22, No. 2, pp. 215–225

Analysis / Contribution: Analyzed Disjoint-Set Union (DSU) with union-by-rank and path compression, proving near-linear O(α(N)) amortized operational bounds for connected components.

Drawbacks: (1) Symmetric / Undirected only: merges components bidirectionally, failing to enforce directed signal propagation (driver-to-load). (2) Irreversible unions: cannot easily split wire nets upon wire deletion.

REVIEW POINT 3

3. Block Diagram & Algorithmic Novelty

System Block Diagram: 4-Stage Architecture

▼ Module 1: Circuit Graph Model

Directed Graph G = (V, E) with Adjacency List. Gates represent vertices; wires represent directed edges. Fast O(1) hash maps store gate types, pins, and in-degrees.

▼ Module 2: Cycle Detection Engine

Tarjan's 3-Color DFS (White=Unvisited, Gray=Visiting, Black=Done). Traps invalid back-edges (feedback loops) and halts simulation with clear error reports.

▼ Module 3: Evaluation Order Scheduler

Kahn's Topological Sort (In-Degree Queue BFS). Derives strict execution order guaranteeing all upstream gate dependencies are evaluated first.

▼ Module 4: Logic Execution & Verification

Boolean Gate Switch Engine + Bitwise Truth Table Generator. Propagates logic signals level-by-level across all 2^N input vectors.

Algorithmic Novelty & Contribution

Dual-Phase Graph Validation: Decouples cycle localization from evaluation scheduling. Uses 3-color DFS specifically to isolate and report offending feedback edges, then uses Kahn's algorithm for deterministic levelized scheduling.

Strict Dependency Invariance: Guarantees zero race conditions. In combinational circuits, every gate evaluates strictly after all its upstream driver inputs are stabilized in O(V + E) time.

Fast O(1) Hash Map Gate Indexing: Provides instantaneous retrieval of gates, netlists, and pin states by name, eliminating O(V) scan overheads during multi-gate propagation.

Automated Exhaustive Verification: Bitwise simulation engine generates full truth tables across all 2^N input permutations, catching logic bugs without physical test vectors.

REVIEW POINT 4 (PART 1)

4. Algorithms / Pseudocode — Graph Model & Cycle Detection

Module 1: Graph Representation

Adjacency list & in-degree table initialization

```
Algorithm BuildCircuitGraph(Gates V, Wires E):
1:  adjList  = HashMap<GateID, List<GateID>>()
2:  inDegree = HashMap<GateID, Integer>()
3:  For each gate g in V:
4:      adjList[g.id] = empty_list()
5:      inDegree[g.id] = 0
6:  For each wire (u, v) in E:
7:      // u is driver gate, v is load gate
8:      adjList[u].append(v)
9:      inDegree[v] = inDegree[v] + 1
10: Return (adjList, inDegree)

Time Complexity:  O(V + E)
Space Complexity: O(V + E)
```

Module 2: Cycle Detection (Tarjan's 3-Color DFS)

Detects feedback loops via back-edges

```
Algorithm HasCycleDFS(u, color, adjList):
1:  color[u] = GRAY  // In recursion stack
2:  For each neighbor v in adjList[u]:
3:      If color[v] == GRAY:
4:          RecordCycleBackEdge(u, v)
5:          Return TRUE  // Cycle detected!
6:      If color[v] == WHITE:
7:          If HasCycleDFS(v, color, adjList) == TRUE:
8:              Return TRUE
9:  color[u] = BLACK // Fully explored
10: Return FALSE

Time Complexity:  O(V + E)
Space Complexity: O(V) recursion stack
```

REVIEW POINT 4 (PART 2)

4. Algorithms / Pseudocode — Scheduling & Simulation

Module 3: Evaluation Scheduler (Kahn's Algorithm)

Topological sort via in-degree BFS queue

```
Algorithm KahnTopologicalSort(V, inDegree, adjList):
1:  Q = EmptyQueue()
2:  For each gate g in V:
3:      If inDegree[g] == 0: Q.push(g)
4:  evalOrder = []
5:  While Q is not empty:
6:      u = Q.pop()
7:      evalOrder.append(u)
8:      For each v in adjList[u]:
9:          inDegree[v] = inDegree[v] - 1
10:         If inDegree[v] == 0: Q.push(v)
11: Return evalOrder

Time Complexity:  O(V + E)
Space Complexity: O(V) queue & order array
```

Module 4: Logic Execution & Verification

Topological propagation & truth table generator

```
Algorithm SimulateCircuit(evalOrder, inputVector):
1:  values = HashMap<GateID, Boolean>()
2:  For each g in evalOrder:
3:      If g.type == INPUT:
4:          values[g] = inputVector[g.id]
5:      Else:
6:          inVals = [values[p] for p in g.predecessors]
7:          values[g] = EvaluateGate(g.type, inVals)
8:  Return values

Algorithm GenerateTruthTable(N_inputs, evalOrder):
1:  For row = 0 to (2^N - 1):
2:      inputVector = DecodeBits(row, N_inputs)
3:      outputRow = SimulateCircuit(evalOrder, inputVector)
4:      PrintTruthTableRow(inputVector, outputRow)
```

REVIEW POINT 5

5. Dataset — IEEE ISCAS '85 Benchmark Circuit Suite

Benchmark Suite & Repository Links

Dataset Name: IEEE ISCAS '85 Benchmark Suite (International Symposium on Circuits and Systems).

Industry Standard: Universally recognized standard benchmark in Electronic Design Automation (EDA) and academic CAD research for combinational circuit validation.

Repository Link 1 (Primary): University of Tennessee EECS CAD Lab: https://web.eecs.utk.edu/~troy/benchmark/

Repository Link 2 (Mirror): Czech Technical University (FEE CTU Benchmarks): https://ddd.fit.cvut.cz/prj/Benchmarks/

Netlist Format (.bench): Human-readable ASCII defining Primary Inputs (INPUT), Primary Outputs (OUTPUT), and gate connection equations (e.g., `10 = NAND(1, 3)`).

Benchmark Circuits & Project Role

c17 (6 Gates, 5 Inputs, 2 Outputs): Smallest ISCAS netlist; used for fundamental algorithmic sanity checks and truth table validation across all 32 combinations.

c432 (160 Gates, 36 Inputs, 7 Outputs): 27-channel interrupt controller; tests topological scheduling performance on medium-scale fanout networks.

c499 (202 Gates, 41 Inputs, 32 Outputs): 32-bit single-error-correcting circuit (SEC-DED); verifies deep logic paths.

c880 (383 Gates, 60 Inputs, 26 Outputs): 8-bit Arithmetic Logic Unit (ALU); evaluates asymptotic time complexity and memory scaling on 380+ nodes.

Project Role: Validates cycle detection correctness, confirms logic outputs against reference truth tables, and demonstrates linear O(V + E) scalability.

CONCLUSION

Summary of First Review Deliverables & Alignment

Review Alignment Matrix

✓ **1. Problem Statement & Need:** Demonstrated physical breadboard constraints (wiring clutter, IC damage, no automated validation) and established the need for a software DAG simulator.

✓ **2. Literature Survey:** Analyzed 3 landmark papers (Kahn 1962, Tarjan 1972 DFS, Tarjan 1975 DSU) and clearly identified their algorithmic drawbacks.

✓ **3. Block Diagram & Novelty:** Presented a 4-module architecture featuring dual-phase graph validation (DFS cycle isolation + Kahn's levelized scheduling).

✓ **4. Algorithms & Pseudocode:** Provided formal pseudocode and asymptotic analysis for all 4 modules (Graph Builder, 3-Color DFS, Kahn's BFS, Logic Simulation).

✓ **5. Dataset & Benchmark:** Documented the official IEEE ISCAS '85 suite with university download links and verified c17 truth table correctness.

Thank You. Open for Questions from the Review Committee.